

Cahier des Charges — Projet de Développement Logiciel (HAX712X)

PyArena : Simulateur Stochastique, Génie Logiciel et Dashboard Décisionnel

Master 1 Sciences des Données (SSD) — Université de Montpellier

2026-09-11

Table des matières

1	Présentation Générale du Projet	2
1.1	Objectifs Pédagogiques	2
1.2	Organisation	2
2	Modèle Mathématique et Règles Métier du Combat	3
2.1	Caractéristiques et Normalisation des Combattants	3
2.2	Matrice des Affinités Élémentaires	3
2.3	Algorithme de Résolution d'un Tour	4
2.3.1	1. Détermination de l'initiative	4
2.3.2	2. Calcul stochastique des dégâts	4
2.3.3	3. Application des dégâts et terminaison	4
3	Architecture Logicielle et Standards de Développement	4
3.1	Arborescence Imposée du Dépôt	4
3.2	Règles de Programmation Orientée Objet (POO)	5
3.3	Packaging et Intégration Continue (CI)	6
4	Pipeline de Données, Cache Local et Simulation	6
4.1	Récupération API et Stratégie de Cache Hors-Ligne	6
4.2	Moteur de Simulation Monte-Carlo Parallélisé	6
5	Spécifications du Tableau de Bord Streamlit	7
5.1	Module 1 : Simulateur de Duel et Télémétrie en Direct	7
5.2	Module 2 : Catalogue et Analyse Exploratoire des Données	7
5.3	Module 3 : Cartographie Géospatiale des Arènes d'Occitanie	8
6	Spécifications Exhaustives des 10 Extensions Thématiques	8
7	Spécifications Exhaustives des 10 Extensions Thématiques	8
7.0.1	Extension 1 : Objets Passifs et Équipements Stratégiques	8
7.0.2	Extension 2 : Combats d'Équipes 3 contre 3 avec Relais (Tag-Team)	9
7.0.3	Extension 3 : Matrice Complète de Méta-Jeu en Round-Robin	9
7.0.4	Extension 4 : Choix Stratégique d'Attaques (Précision vs Puissance)	10
7.0.5	Extension 5 : Système Stochastique d'Altérations d'État	10

7.0.6	Extension 6 : Gestion des Points de Compétence (PP) et Action Lutte	11
7.0.7	Extension 7 : Accélération Vectorisée et Profilage Numérique	11
7.0.8	Extension 8 : Système Dynamique de Niveaux et Courbe de Bascule	12
7.0.9	Extension 9 : La Route des Champions (Occitanie - Progression sous Contrainte) .	12
7.0.10	Extension 10 : Reconstitution Pas-à-Pas et Export de Combat (Replay Interactif)	13
8	Politique de Bonus (Valorisation du Dépassement Technique)	14
8.1	Modalités d'Attribution	14
8.2	Exemples d'Initiatives Reconnues pour le Bonus	14
9	Travail Collaboratif, Gestion de Version et Intégrité	14
9.1	Bonnes Pratiques Git Imposées	14
9.2	Document Contractuel : CONTRIBUTIONS.md	15
9.3	Contrôle Automatisé du Plagiat et de l'Histoire	15
10	Modalités d'Évaluation et Grilles de Notation	15
10.1	Note 1 : Code, Architecture et Produit (/20 — Note de Groupe, Coeff 3)	15
10.2	Note 2 : Soutenance Orale et Maîtrise Individuelle (/20 — Note Individuelle, Coeff 2) . .	16
11	Calendrier Prévisionnel et Jalons de Développement	17

1 Présentation Générale du Projet

Le projet de l'UE Développement Logiciel (HAX712X) constitue le fil conducteur pratique du semestre pour la promotion de **Master 1 Sciences des Données (SSD)**.

L'objectif est de concevoir, développer, tester, documenter et déployer une bibliothèque logicielle complète en Python modélisant une arène de combat au tour par tour (**moteur headless**), couplée à un pipeline d'ingestion de données et à un **tableau de bord analytique interactif (Streamlit)**.

1.1 Objectifs Pédagogiques

1. **Conception logicielle industrielle** : Programmation orientée objet stricte, modularité, encapsulation, gestion rigoureuse des erreurs et packaging standard (`pyproject.toml`).
2. **Assurance qualité** : Tests unitaires automatisés (`pytest`), mesure de couverture de code et intégration continue (GitHub Actions).
3. **Calcul intensif et Data Science** : Simulation stochastique de Monte-Carlo, parallélisation multi-cœurs (`multiprocessing` / `joblib`), ingestion d'API Web et mise en cache locale.
4. **Visualisation avancée** : Exploration de données (Plotly), cartographie géographique interactive (Folium / GeoPandas) et déploiement d'une application web décisionnelle.

1.2 Organisation

- Le projet s'effectue obligatoirement en **groupes de 3 étudiants** (exceptionnellement un groupe de 4 en fonction de l'effectif total).
- Le moteur de simulation, l'ingestion API, les tests unitaires et le dashboard constituent un **socle commun obligatoire**.

- Chaque groupe choisit **une extension thématique exclusive** parmi les 10 proposées afin d’approfondir un volet algorithmique ou fonctionnel précis.

2 Modèle Mathématique et Règles Métier du Combat

Le système dynamique modélisé est un duel discret au tour par tour entre deux entités. Le modèle est entièrement formel et autosuffisant : **aucune connaissance préalable de jeux vidéo (Pokemon) n’est requise.**

2.1 Caractéristiques et Normalisation des Combattants

Chaque combattant C est modélisé par un vecteur d’état à sept dimensions :

$$C = \langle \text{id}, \text{nom}, T, \text{PV}_{\max}, \text{PV}, A, D, V \rangle$$

- $\text{id} \in \llbracket 1, 151 \rrbracket$: identifiant officiel dans la PokéAPI (première génération).
- $\text{nom} \in \mathbb{S}$: identifiant textuel unique (ex. : "Pikachu", "Charizard").
- $T \in \mathcal{T}$: type élémentaire appartenant à $\mathcal{T} = \{\text{Normal}, \text{Feu}, \text{Eau}, \text{Plante}, \text{Électrik}, \dots\}$.
- Les caractéristiques opérationnelles sont normalisées au **Niveau 50 standard** à partir des statistiques de base brutes ($\text{Stat}_{\text{base}}$) récupérées sur l’API :

$$\text{PV}_{\max} = \text{PV}_{\text{base}} + 60$$

$$A = A_{\text{base}} + 5, \quad D = D_{\text{base}} + 5, \quad V = V_{\text{base}} + 5$$

- $\text{PV} \in \llbracket 0, \text{PV}_{\max} \rrbracket$: points de vie courants du combattant.

Si $\text{PV} = 0$, l’entité est déclarée **K.-O.** (*fainted*) et ne peut plus réaliser aucune action.

2.2 Matrice des Affinités Élémentaires

Le coefficient multiplicateur $M(T_{\text{att}}, T_{\text{déf}}) \in \{0.5, 1.0, 2.0\}$ modélise l’avantage ou le désavantage élémentaire :

Type Attaquant (T_{att})	Type Défenseur ($T_{\text{déf}}$)	Normal	Feu	Eau	Plante	Électrik
Normal		1.0	1.0	1.0	1.0	1.0
Feu		1.0	0.5	0.5	2.0	1.0
Eau		1.0	2.0	0.5	0.5	1.0
Plante		1.0	0.5	2.0	0.5	1.0
Électrik		1.0	1.0	2.0	0.5	0.5

Un fichier `data/types.csv` est fourni pour charger cette matrice de manière automatisée dans le package.

2.3 Algorithme de Résolution d'un Tour

Soient deux combattants C_1 et C_2 . Le duel se décompose en tours séquentiels successifs :

2.3.1 1. Détermination de l'initiative

- Le combattant possédant la vitesse V la plus élevée frappe en premier.
- En cas d'égalité stricte ($V_1 = V_2$), le premier attaquant est désigné par un tirage de Bernoulli $B \sim \mathcal{B}(0.5)$.

2.3.2 2. Calcul stochastique des dégâts

L'attaquant actif assène une attaque standard de puissance nominale $P = 50$.

La formule fermée des dégâts réels infligés est :

$$\text{Dégâts} = \max \left(1, \left\lfloor \left(\frac{22 \times P \times \frac{A_{\text{att}}}{D_{\text{déf}}}}{50} + 2 \right) \times M(T_{\text{att}}, T_{\text{déf}}) \times U \times K \right\rfloor \right)$$

avec : * $\lfloor x \rfloor$: partie entière inférieure. * $M(T_{\text{att}}, T_{\text{déf}})$: multiplicateur issu de la matrice des types. * U : variable aléatoire modélisant la variabilité du coup, $U \sim \mathcal{U}([0.85, 1.00])$. * K : multiplicateur de coup critique gouverné par une variable de Bernoulli $X_{\text{crit}} \sim \mathcal{B}(0.10)$:

$$K = \begin{cases} 1.5 & \text{si } X_{\text{crit}} = 1 \text{ (probabilité 10\%)} \\ 1.0 & \text{sinon} \end{cases}$$

2.3.3 3. Application des dégâts et terminaison

$$PV_{\text{déf}} \leftarrow \max(0, PV_{\text{déf}} - \text{Dégâts})$$

- Si $PV_{\text{déf}} = 0$: le défenseur est K.-O., le combat prend fin **immédiatement**. Le défenseur ne riposte pas.
- Si $PV_{\text{déf}} > 0$: le défenseur devient attaquant et exécute à son tour les étapes 2 et 3.

3 Architecture Logicielle et Standards de Développement

3.1 Arborescence Imposée du Dépôt

L'organisation des répertoires doit impérativement respecter le schéma suivant :

```

pyarena/
  pyproject.toml          # Configuration du package, pytest, linters et dépendances
  README.md              # Instructions d'installation et de reproduction
  CONTRIBUTIONS.md       # Tableau contractuel de contribution individuelle
  data/
    types.csv            # Matrice des coefficients élémentaires
    arenes_occitanie.geojson # Données spatiales régionales
    cache/               # Fichiers JSON locaux de l'API (ignorés par Git)
  src/
    pyarena/
      __init__.py        # Export des classes et fonctions publiques
      core/
        __init__.py
        exceptions.py    # Classes d'erreurs maison
        models.py        # Classes Fighter, Team, BattleResult
        engine.py        # Moteur d'exécution des tours et du combat
      io/
        __init__.py
        fetcher.py       # Client HTTP PokéAPI avec stratégie de cache
        loader.py        # Parsers CSV et GeoJSON
      simulation/
        __init__.py
        monte_carlo.py   # Moteur Monte-Carlo parallélisé (joblib/multiprocessing)
      extensions/
        __init__.py      # Emplacement unique du code de l'extension choisie
  tests/
    __init__.py
    test_models.py       # Tests unitaires des entités et de l'encapsulation
    test_engine.py       # Tests déterministes du calcul des dégâts
    test_simulation.py    # Tests de reproductibilité et de parallélisation
    test_extension.py     # Tests unitaires spécifiques à l'extension
  app/
    main.py              # Point d'entrée de l'application Streamlit
    pages/               # Pages multipages Streamlit
    utils.py             # Fonctions d'affichage graphique Plotly/Folium

```

3.2 Règles de Programmation Orientée Objet (POO)

1. Abstraction :

- Présence obligatoire d'une classe abstraite `BaseFighter` héritant de `abc.ABC`.
- Définition explicite des méthodes abstraites via `@abstractmethod` :
 - `take_damage(self, amount: int) -> None`
 - `is_alive(self) -> bool`
 - `reset_stats(self) -> None`

2. Encapsulation stricte :

- Les attributs critiques (dont les points de vie) doivent être privés ou protégés et exposés via des propriétés `@property`.

- Le setter associé à la santé doit empêcher toute affectation d'une valeur négative ou supérieure à $PV_{\max}$.
3. **Méthodes spéciales (dunder methods) :**
- Implémenter obligatoirement `__repr__` (représentation technique complète), `__str__` (affichage convivial) et `__eq__` (égalité structurelle basée sur l'identifiant).
4. **Hiérarchie d'exceptions maison :**
- Définir une exception de base `PyArenaException(Exception)` dans `exceptions.py`.
 - Implémenter au minimum :
 - `FighterFaintedError` : levée lorsqu'un combattant K.-O. tente d'agir.
 - `InvalidStatError` : levée si une statistique d'entrée est hors des bornes autorisées.
 - `DataFetchError` : levée si l'API est injoignable et qu'aucun cache local n'existe.

3.3 Packaging et Intégration Continue (CI)

- Le paquet doit être installable en mode développement à la racine du dépôt :

```
pip install -e .
```
- Un workflow GitHub Actions (`.github/workflows/ci.yml`) doit s'exécuter à chaque commit poussé sur `main` et à chaque Pull Request :
 1. Vérification du formatage et du style avec `ruff check` ou `flake8`.
 2. Exécution de l'intégralité des tests avec `pytest`.
 3. Vérification du seuil de couverture de code : **la couverture mesurée par `pytest-cov` sur le package `pyarena` doit être supérieure ou égale à 70 %**.

4 Pipeline de Données, Cache Local et Simulation

4.1 Récupération API et Stratégie de Cache Hors-Ligne

- Le module `fetcher.py` doit implémenter une classe `DataFetcher` interrogeant l'API publique ouverte [<https://pokeapi.co/api/v2/pokemon/>] (<https://pokeapi.co/api/v2/pokemon/{id}>) pour les 151 premières créatures (`id ∈ [1, 151]`).
- **Mise en cache locale obligatoire :** Tout appel réseau réussi enregistre immédiatement le fichier JSON sous `data/cache/{id}.json`.
- Lors d'une demande ultérieure, le système charge directement le fichier local sans déclencher de requête HTTP.
- Le dossier `data/cache/` doit figurer dans le fichier `.gitignore` : les données brutes ne doivent pas être versionnées sur Git.

4.2 Moteur de Simulation Monte-Carlo Parallélisé

- La fonction de combat élémentaire `simulate_battle(f1, f2, seed=None)` doit permettre l'injection d'une graine aléatoire (`seed`) pour garantir la reproductibilité parfaite d'une confrontation.
- Le module `monte_carlo.py` doit proposer une fonction :

```
def run_monte_carlo(
    f1: BaseFighter,
    f2: BaseFighter,
    n_simulations: int = 5000,
    n_jobs: int = -1
) -> dict:
```

- **Parallélisation** : L'exécution des $n_{\text{simulations}}$ duels doit être distribuée sur l'ensemble des cœurs CPU de la machine hôte (`multiprocessing.Pool` ou `joblib.Parallel`).
- **Mesures retournées** :

1. Probabilité de victoire estimée de C_1 :

$$\hat{p} = \frac{\text{Nombre de victoires de } C_1}{n_{\text{simulations}}}$$

2. Intervalle de confiance asymptotique à 95 % :

$$\text{IC}_{0.95}(\hat{p}) = \left[\hat{p} - 1.96 \sqrt{\frac{\hat{p}(1-\hat{p})}{n_{\text{simulations}}}}, \hat{p} + 1.96 \sqrt{\frac{\hat{p}(1-\hat{p})}{n_{\text{simulations}}}} \right]$$

3. Distribution du nombre de tours jusqu'au terme du combat (moyenne, médiane, écart-type).

5 Spécifications du Tableau de Bord Streamlit

L'application Streamlit (`app/main.py`) est un outil décisionnel interactif comprenant obligatoirement les trois modules suivants :

5.1 Module 1 : Simulateur de Duel et Télémétrie en Direct

- Deux listes déroulantes permettent de sélectionner les combattants A et B parmi les 151 disponibles.
- Affichage immédiat des sprites officiels (PNG transparents issus du cache ou de l'URL) et de leurs statistiques de niveau 50.
- Curseur interactif pour régler le nombre de simulations ($100 \leq N \leq 10\,000$).
- Restitution post-calcul :
 - Winrates comparés avec barres de progression.
 - Intervalle de confiance à 95 % affiché numériquement.
 - Histogramme Plotly interactif de la distribution du nombre de tours.
 - Journal textuel pas-à-pas (*log*) d'un match témoin généré avec graine fixée.

5.2 Module 2 : Catalogue et Analyse Exploratoire des Données

- Tableau de données interactif avec pagination et filtres dynamiques (par type, par plage de PV ou d'attaque).
- Graphique de dispersion interactif Plotly : *Attaque* en abscisse, *Défense* en ordonnée, taille des marqueurs proportionnelle aux PV, couleur assignée selon le *Type*.
- Boîtes à moustaches (*boxplots*) comparant la distribution de la vitesse entre les 5 familles élémentaires.

5.3 Module 3 : Cartographie Géospatiale des Arènes d'Occitanie

- Exploitation du fichier `data/arenas_occitanie.geojson` regroupant 30 sites régionaux (Montpellier, Sète, Nîmes, Toulouse, Perpignan...).
 - Affichage de la carte dynamique via `streamlit-folium`.
 - Marqueurs géoréférencés colorés selon le type élémentaire de l'arène.
 - Clic sur un marqueur : ouverture d'un pop-up affichant le nom de l'arène, son champion, son sprite officiel et ses statistiques.
-

6 Spécifications Exhaustives des 10 Extensions Thématiques

Chaque groupe développe **une seule** extension. L'extension doit obligatoirement comporter : 1. Un module dédié dans `src/pyarena/extensions/`. 2. Une suite de tests unitaires dédiée dans `tests/test_extension.py`. 3. Un onglet ou composant dédié dans l'application Streamlit.

7 Spécifications Exhaustives des 10 Extensions Thématiques

Chaque groupe développe **une seule** extension parmi la liste ci-dessous. Toutes les extensions sont calibrées pour représenter un volume de travail équivalent (environ 100 à 150 lignes de code métier, une suite de tests unitaires dédiée et un composant interactif dans Streamlit).

Aucune extension ne requiert d'algorithme complexe hors programme : elles s'appuient exclusivement sur la modélisation orientée objet, des calculs de probabilités élémentaires, des boucles de balayage ou des manipulations de données tabulaires/spatiales.

7.0.1 Extension 1 : Objets Passifs et Équipements Stratégiques

- **Problématique jeu** : Évaluer comment l'attribution d'un objet passif peut compenser un déficit initial de statistiques ou modifier radicalement l'issue d'un duel.
- **Spécifications logicielles** :
 - Créer une classe de base abstraite `BaseItem` dans `src/pyarena/extensions/items.py` avec les méthodes :
 - `apply_passive(self, fighter: BaseFighter) -> None` (ajustement de statistiques au début du match)
 - `on_damage_received(self, incoming_damage: int, current_hp: int) -> int` (altération des dégâts subis)
 - `on_turn_end(self, fighter: BaseFighter) -> None` (effets de régénération en fin de tour)
 - Implémenter obligatoirement 4 objets concrets :

1. *Ceinture Force* : si $PV = PV_{\max}$ et que les dégâts entrants sont mortels, laisse le combattant à $PV = 1$ (usage unique par match).
 2. *Baie Citrus* : dès que $PV \leq 0,5 \times PV_{\max}$, rend immédiatement $\lfloor 0,25 \times PV_{\max} \rfloor$ PV (consommable, usage unique).
 3. *Orbe Vie* : augmente les dégâts infligés par le porteur de 30 %, mais lui retire $\lfloor 0,10 \times PV_{\max} \rfloor$ PV après chaque frappe assénée.
 4. *Restes* : régénère $\lfloor \frac{PV_{\max}}{16} \rfloor$ PV à la fin de chaque tour.
- **Tests unitaires (tests/test_extension.py) :**
 - Vérifier que la Ceinture Force s’active bien sur une frappe létale si les PV sont pleins, mais pas si le combattant a déjà subi des dégâts préalables.
 - Vérifier que la Baie Citrus ne s’active pas au-dessus du seuil de 50 % de PV.
 - Contrôler le bon calcul des dégâts de contrecoup de l’Orbe Vie.
 - **Intégration Streamlit :** Menus déroulants permettant d’équiper un objet sur le combattant *A* et/ou *B*, et affichage du delta de winrate (+X %) induit par chaque objet.
-

7.0.2 Extension 2 : Combats d’Équipes 3 contre 3 avec Relais (Tag-Team)

- **Problématique jeu :** Modéliser un format de match en relais et déterminer si l’ordre d’entrée en piste (*lead*) influence statistiquement la victoire finale.
 - **Spécifications logicielles :**
 - Créer une classe `Team` dans `src/pyarena/extensions/teams.py` gérant une file ordonnée de 3 combattants.
 - Créer une classe `TeamBattle` : les premiers combattants de chaque équipe s’affrontent. Dès qu’un combattant tombe K.-O., le suivant dans la file entre immédiatement dans l’arène. Le combattant adverse conserve ses PV entamés (pas de régénération entre les duels).
 - Condition de fin : la confrontation s’arrête dès que les 3 membres d’une équipe sont tous K.-O.
 - **Tests unitaires (tests/test_extension.py) :**
 - Vérifier le passage automatique au combattant numéro 2 lors du premier K.-O.
 - Vérifier que le combattant restant conserve exactement ses PV résiduels à l’entrée du combattant adverse suivant.
 - Valider l’arrêt de la partie au troisième K.-O.
 - **Intégration Streamlit :** Sélecteur de 3 combattants par équipe, affichage dynamique des 6 jauges de santé et comparaison graphique du winrate selon que l’on inverse l’ordre d’apparition des membres d’une équipe.
-

7.0.3 Extension 3 : Matrice Complète de Méta-Jeu en Round-Robin

- **Problématique jeu :** Identifier si un panel de combattants présente des relations circulaires intransitives (type « Pierre-Feuille-Ciseaux ») ou s’il existe une créature dominante dans tous les cas de figure.
- **Spécifications logicielles :**
 - Créer un module `src/pyarena/extensions/round_robin.py`.
 - Isoler une sélection fixe de 10 combattants représentatifs aux types élémentaires variés.

- Automatiser le calcul des duels croisés : $\frac{10 \times 9}{2} = 45$ confrontations distinctes, chacune simulée sur $N = 1\,500$ itérations parallélisées.
 - Construire la matrice d'adjacence 10×10 où la cellule (i, j) consigne l'estimation ponctuelle $\hat{p}(C_i \text{ bat } C_j)$.
 - **Tests unitaires (tests/test_extension.py) :**
 - Vérifier les propriétés de cohérence de la matrice : $M_{i,i} = 0,5$ et pour tout $i \neq j$, $M_{i,j} + M_{j,i} = 1,0$ (à tolérance machine près).
 - **Intégration Streamlit :** Heatmap interactive Plotly de la matrice 10×10 , accompagnée du tableau récapitulatif classant les 10 combattants selon leur score moyen cumulé.
-

7.0.4 Extension 4 : Choix Stratégique d'Attaques (Précision vs Puissance)

- **Problématique jeu :** Comparer la fiabilité d'une stratégie de jeu prudente (dégâts modérés constants) face à une stratégie agressive reposant sur des attaques puissantes mais imprécises.
 - **Spécifications logicielles :**
 - Modifier la classe `Fighter` dans `src/pyarena/extensions/moves.py` pour lui conférer deux capacités :
 1. *Frappe Sûre* : Puissance $P = 40$, Précision $\alpha = 1,00$ (réussite garantie).
 2. *Frappe Risquée* : Puissance $P = 85$, Précision $\alpha = 0,65$ (35 % de chances d'échouer complètement et de n'infliger aucun dégât).
 - Implémenter 3 règles de décision sélectionnables pour l'attaquant :
 - *Stratégie Prudente* : utilise systématiquement la Frappe Sûre.
 - *Stratégie Agressive* : utilise systématiquement la Frappe Risquée.
 - *Stratégie Gloutonne (Espérance)* : calcule à chaque tour les dégâts moyens espérés de chaque coup $E[\text{Dégâts}] = \alpha \times \text{Dégâts}$ et choisit la frappe qui maximise ce gain immédiat.
 - **Tests unitaires (tests/test_extension.py) :**
 - Valider sur 10 000 lancers simulés que la fréquence d'échec de la Frappe Risquée est bien conforme à $0,35 \pm 0,015$.
 - Vérifier qu'une attaque ratée n'inflige aucun dégât et ne déclenche aucun coup critique.
 - **Intégration Streamlit :** Interface permettant d'assigner une stratégie à chaque joueur et affichage superposé des distributions de victoires obtenues.
-

7.0.5 Extension 5 : Système Stochastique d'Altérations d'État

- **Problématique jeu :** Mesurer à quel point des altérations de statut infligées aléatoirement peuvent déstabiliser un duel et prolonger la durée des matches.
- **Spécifications logicielles :**
 - Définir une énumération `Status(Enum)` dans `src/pyarena/extensions/statuses.py` : `NONE`, `PARALYSIS`, `BURN`.
 - Chaque coup porté a une probabilité $p = 0,15$ d'induire une altération d'état sur la cible (si elle est indemne) :
 - *Paralysie* : divise définitivement la statistique de vitesse par deux ($V \leftarrow \lfloor V/2 \rfloor$). De plus, au début de chaque tour, le combattant a 25 % de chances d'être tétanisé et d'annuler son attaque.

- *Brûlure* : divise l'Attaque par deux ($A \leftarrow \lfloor A/2 \rfloor$). À la fin de chaque tour, le porteur perd $\lfloor \frac{PV_{\max}}{16} \rfloor$ PV résiduels.
 - **Tests unitaires (tests/test_extension.py) :**
 - Vérifier que la vitesse ou l'attaque est effectivement divisée par deux lors de l'application du statut.
 - Valider la non-cumulabilité des statuts majeurs : une entité déjà paralysée ne peut pas être brûlée.
 - **Intégration Streamlit :** Visualisation par histogrammes Plotly comparant l'espérance et la variance du nombre de tours jusqu'au K.-O. avec et sans altérations de statut.
-

7.0.6 Extension 6 : Gestion des Points de Compétence (PP) et Action Lutte

- **Problématique jeu :** Introduire la gestion des ressources en combat d'usure en limitant le nombre de frappes puissantes disponibles avant le recours à l'action de secours.
 - **Spécifications logicielles :**
 - Créer une classe `Move` dans `src/pyarena/extensions/skills.py` associant une puissance, un type et un capital de Points de Puissance (PP).
 - Doter chaque combattant de deux capacités :
 1. *Capacité Majeure* : Puissance $P = 75$, mais limitée à $PP_{\max} = 5$ utilisations.
 2. *Capacité Standard* : Puissance $P = 45$, dotée de $PP_{\max} = 15$ utilisations.
 - Le combattant consomme en priorité sa capacité majeure, puis sa capacité standard lorsque les PP sont épuisés.
 - Dès que le stock total de PP tombe à 0, le combattant est contraint d'utiliser l'action par défaut *Lutte* : puissance fixe $P = 50$, et subit en contrecoup immédiat une perte de PV égale à $\lfloor 0,25 \times \text{Dégâts infligés} \rfloor$.
 - **Tests unitaires (tests/test_extension.py) :**
 - Vérifier la décrémentation stricte des compteurs de PP à chaque utilisation.
 - Valider le basculement forcé vers l'action Lutte dès que l'ensemble des PP est épuisé.
 - Contrôler le calcul arithmétique de la perte de points de vie par contrecoup.
 - **Intégration Streamlit :** Jauges visuelles d'épuisement des réserves de PP en temps réel et suivi statistique de la fréquence d'apparition du mode Lutte lors des matchs longs.
-

7.0.7 Extension 7 : Accélération Vectorisée et Profilage Numérique

- **Problématique jeu :** Identifier les goulots d'étranglement de l'implémentation objet en temps d'exécution et proposer une réécriture mathématique vectorisée pour traiter de très gros volumes de simulations.
- **Spécifications logicielles :**
 - Profiler l'exécution de 10 000 combats en Python objet standard avec le module `cProfile` pour chiffrer le temps passé dans chaque méthode.
 - Concevoir dans `src/pyarena/extensions/fast_sim.py` une version optimisée de la boucle de combat :

- Les entités sont représentées par de simples tableaux NumPy 1D d'entiers et de flottants au lieu d'instances de classes complexes.
 - Utilisation de générateurs de nombres aléatoires vectorisés (`numpy.random.default_rng`) ou de fonctions compilées avec le décorateur `@njit` de **Numba**.
 - Construire un banc d'essai comparant rigoureusement le temps d'exécution sur des tailles d'échantillons $N \in [10^2, 10^5]$.
 - **Tests unitaires (`tests/test_extension.py`) :**
 - Test de conformité statistique validant que la fonction rapide et la version orientée objet produisent des winrates identiques à tolérance d'échantillonnage près.
 - **Intégration Streamlit :** Graphique interactif du facteur d'accélération ($speedup = \frac{T_{objet}}{T_{optimisé}}$) affiché sur le dashboard en fonction du volume de simulations N .
-

7.0.8 Extension 8 : Système Dynamique de Niveaux et Courbe de Bascule

- **Problématique jeu :** Déterminer précisément à partir de quel différentiel de niveau un combattant intrinsèquement défavorisé (statistiques faibles ou type élémentaire désavantageux) parvient à renverser les probabilités face à un adversaire supérieur.
- **Spécifications logicielles :**
 - Créer un module `src/pyarena/extensions/levels.py` permettant d'instancier un combattant à un niveau arbitraire $L \in \llbracket 1, 100 \rrbracket$.
 - Implémenter le recalcul dynamique des caractéristiques selon les formules officielles :

$$PV(L) = \left\lfloor \frac{2 \times PV_{base} \times L}{100} \right\rfloor + L + 10$$

$$Stat(L) = \left\lfloor \frac{2 \times Stat_{base} \times L}{100} \right\rfloor + 5 \quad (\text{pour } A, D, V)$$

- Implémenter une fonction de balayage `find_turnover_level(f1, f2_lv150, step=5)` qui évalue le winrate de C_1 pour des niveaux progressifs $L \in \{5, 10, 15, \dots, 100\}$ face à un adversaire fixé au niveau 50.
 - Déterminer le *Niveau Critique* L^* correspondant au premier niveau où $\hat{p}(L^*) \geq 0,50$.
 - **Tests unitaires (`tests/test_extension.py`) :**
 - Valider l'exactitude numérique des statistiques calculées aux niveaux bornes $L = 1$, $L = 50$ et $L = 100$ à partir de valeurs de test vérifiées.
 - Vérifier que les statistiques sont des fonctions strictement croissantes du niveau L .
 - **Intégration Streamlit :** Curseur de niveau dynamique et courbe Plotly de la fonction de winrate $L \mapsto \hat{p}(L)$ avec mise en évidence du seuil de bascule L^* .
-

7.0.9 Extension 9 : La Route des Champions (Occitanie - Progression sous Contrainte)

- **Problématique jeu :** Modéliser la tournée géographique d'un dresseur devant défier les arènes régionales dans un ordre de difficulté croissant, et mesurer le surcoût kilométrique imposé par cette progression par rapport à un itinéraire géographiquement direct.
- **Spécifications logicielles :**

- Exploiter le fichier `data/arenas_occitanie.geojson` contenant les coordonnées géographiques et les niveaux de difficulté des arènes régionales (échelonnés du niveau 15 au niveau 60).
- Dans `src/pyarena/extensions/routing.py`, implémenter le calcul de la distance géodésique de Haversine entre deux coordonnées (lat, lon) :

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\varphi}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad \text{avec } R = 6\,371 \text{ km}$$

- Calculer et comparer deux itinéraires partant de Montpellier :
 1. *L'Itinéraire des Champions* : visite obligatoire des arènes triées par ordre de niveau croissant.
 2. *L'Itinéraire Naïf le Plus Proche* : visite séquentielle selon la règle gloutonne du point géographiquement le plus proche sans tenir compte du niveau.
- **Tests unitaires (`tests/test_extension.py`) :**
 - Vérifier que la distance de Haversine entre deux points identiques est nulle, et conforme aux distances réelles interurbaines connues (ex. Montpellier–Nîmes $\approx 45 - 55$ km à vol d'oiseau).
 - Valider que chaque ville de la sélection est visitée exactement une fois dans les deux parcours.
- **Intégration Streamlit :** Tracé comparatif des deux tracés sur la carte interactive Folium (`folium.PolyLine` de couleurs distinctes) et affichage métrique de l'écart kilométrique induit par la contrainte de niveau.

7.0.10 Extension 10 : Reconstitution Pas-à-Pas et Export de Combat (Replay Interactif)

- **Problématique jeu :** Permettre l'enregistrement, l'exportation et le visionnage interactif tour par tour d'un match stochastique remarquable sans recalculer la simulation.
- **Spécifications logicielles :**
 - Concevoir dans `src/pyarena/extensions/replayer.py` une structure d'enregistrement capturant l'ensemble des événements du match dans une structure tabulaire ou une liste de dictionnaires : `{"tour": i, "attaquant": str, "cible": str, "degats": int, "critique": bool, "pv_restants_cible": int}`.
 - Implémenter deux fonctions d'entrée/sortie :
 - `export_match_json(battle_log, filepath: str) -> None` : sauvegarde du journal au format JSON.
 - `load_match_json(filepath: str) -> list[dict]` : chargement et validation du schéma d'un journal sauvegardé.
- **Tests unitaires (`tests/test_extension.py`) :**
 - Vérifier que l'export puis le rechargement d'un match redonnent exactement la même séquence de valeurs (test d'aller-retour / sérialisation).
 - Vérifier que le journal consigne bien une décroissance monotone des PV de la cible et s'arrête exactement au tour où une entité atteint 0 PV.
- **Intégration Streamlit :**
 - Composant de téléversement (`st.file_uploader`) permettant de charger un match sauvegardé au format JSON.
 - Lecteur interactif avec curseur temporel et boutons de navigation (« *Tour Précédent* » / « *Tour Suivant* »).
 - Barres de santé animées changeant de couleur selon l'état du combattant (vert > 50%, orange 20 – 50%, rouge < 20%) et boîte de dialogue commentant l'action du tour sélectionné.

8 Politique de Bonus (Valorisation du Dépassement Technique)

Afin d'encourager la curiosité, l'approfondissement algorithmique et les initiatives personnelles, un système de **points de bonus** est prévu.

8.1 Modalités d'Attribution

- **Plafond strict** : Les bonus peuvent rapporter jusqu'à **+2 points** sur la Note Technique de Code (la note finale restant plafonnée à 20/20).
- **Condition impérative d'éligibilité** : Les points de bonus ne sont comptabilisés que si le **socle commun obligatoire** (POO, packaging, tests unitaires avec couverture $\geq 70\%$, intégration continue GitHub Actions et dashboard) et **l'extension attribuée** sont intégralement fonctionnels et exempts de régression le jour du rendu.
- **Documentation obligatoire** : Tout ajout faisant l'objet d'une demande de bonus doit être explicitement répertorié dans une section dédiée du fichier README.md intitulée **## Fonctionnalités Bonus Implémentées**.

8.2 Exemples d'Initiatives Reconnues pour le Bonus

1. **Prise en compte du double type élémentaire (+0,75 pt)** : Intégrer le fait que certaines créatures possèdent deux types combinés (ex. Feu/Vol ou Eau/Sol). Adapter le calcul de dégâts pour multiplier les coefficients d'affinité élémentaire issus des deux types, autorisant des multiplicateurs de dégâts cumulés de $\times 4$ ou de $\times 0,25$.
2. **Matrice officielle complète des 18 types (+0,5 pt)** : Remplacer la matrice simplifiée des éléments par l'ingestion et la gestion rigoureuse de la table officielle complète des 18 types (incluant Spectre, Dragon, Acier, Fée...), avec gestion des immunités strictes (multiplicateur $\times 0,0$).
3. **Arbre de décision tactique (Minimax élémentaire à 2 tours) (+1 pt)** : Pour les extensions impliquant des choix d'actions multiples (Extensions 2 ou 4), concevoir un agent évaluant l'arbre des états possibles à horizon 2 tours afin de sélectionner l'action minimisant la perte maximale de PV face au contre adverse.
4. **Intégration multimédia officielle (Audio & Thèmes) (+0,5 pt)** : Exploiter les champs audio disponibles sur l'API pour jouer les cris officiels des combattants (format OGG/MP3 via `st.audio`) lors du K.-O. ou de l'entrée dans l'arène, et habiller l'application Streamlit d'une charte graphique soignée.
5. **Déploiement public sur le Cloud (+0,5 pt)** : Déployer l'application Streamlit sur l'infrastructure gratuite *Streamlit Community Cloud* et fournir l'URL publique active directement dans l'en-tête du fichier README.md.

9 Travail Collaboratif, Gestion de Version et Intégrité

9.1 Bonnes Pratiques Git Imposées

1. **Protection absolue de la branche main** : Aucun push direct n'est toléré sur `main`.

2. **Workflow par branches et Pull Requests** : Toute nouvelle brique ou correction d'anomalie fait l'objet d'une branche distincte (ex. : `feat/models-validation`, `fix/crit-calc`), soumise à validation par une Pull Request relue par un autre membre du groupe.
3. **Atomicité des commits** : Les commits doivent être ciblés et porter un message explicite respectant la convention standard (`feat: ...`, `fix: ...`, `test: ...`).

9.2 Document Contractuel : CONTRIBUTIONS.md

À la racine du dépôt, le fichier `CONTRIBUTIONS.md` doit consigner précisément l'investissement effectif de chaque membre selon le tableau type :

Membre	Identifiant GitHub	Rôle et Modules Développés	Numéros des PRs Soumises et Revues
Étudiant 1	@login1	Conception du module <code>core/</code> , tests <code>pytest</code>	PRs #1, #4 (Revue : #2, #5)
Étudiant 2	@login2	Client API <code>io/fetcher.py</code> , Dashboard Streamlit	PRs #2, #5 (Revue : #3)
Étudiant 3	@login3	Parallélisation <code>simulation/</code> , Extension choisie	PRs #3, #6 (Revue : #1, #4)

9.3 Contrôle Automatisé du Plagiat et de l'Histoire

- **Analyse de régularité** : La distribution temporelle des commits sera examinée sur GitHub. Tout « commit massif » (injection soudaine de plusieurs centaines de lignes de code en fin de projet) sans historique incrémental fera l'objet d'une pénalité individuelle.
- **Analyse syntaxique structurelle (`copydetect`)** : L'ensemble des répertoires `src/` rendus sera soumis à un contrôle automatisé comparant les arbres syntaxiques abstraits (AST) via l'outil `copydetect`. Les dépôts présentant une similarité non fortuite seront pénalisés.

10 Modalités d'Évaluation et Grilles de Notation

L'évaluation donne lieu à deux notes sur 20 indépendantes : la **Note Technique de Code (coefficient 3)** et la **Note Individuelle de Soutenance (coefficient 2)**.

$$\text{Note Finale} = \frac{3 \times \text{Note Code} + 2 \times \text{Note Soutenance}}{5}$$

10.1 Note 1 : Code, Architecture et Produit (/20 — Note de Groupe, Coeff 3)

Critère	Barème	Éléments évalués concrètement
Architecture Orientée Objet	/ 4	Respect des classes abstraites, encapsulation (<code>@property</code>), gestion des exceptions maison, typage strict.
Packaging & Propreté	/ 2	Fichier <code>pyproject.toml</code> fonctionnel, installation immédiate via <code>pip install -e .</code> , arborescence scrupuleusement respectée.
Tests & Intégration Continue	/ 4	Couverture mesurée par <code>pytest-cov</code> $\geq 70\%$, pertinence des tests aux limites, workflow GitHub Actions au vert (si rouge : -2 points d'office).
Données & Parallélisme	/ 3	Système de cache local opérationnel (mode déconnecté), exécution distribuée de Monte-Carlo (<code>joblib/multiprocessing</code>), intervalles de confiance corrects.
Dashboard Streamlit	/ 4	Réactivité de l'application, présence des sprites officiels, visualisations Plotly pertinentes, carte Folium fonctionnelle.
Réalisation de l'Extension	/ 3	Respect rigoureux du cahier des charges spécifique à l'extension attribuée et couverture par ses tests unitaires.

10.2 Note 2 : Soutenance Orale et Maîtrise Individuelle (/20 — Note Individuelle, Coeff 2)

Chaque groupe dispose de **20 minutes d'évaluation** décomposées en : * 10 minutes d'exposé et de démonstration applicative en direct. * 10 minutes de questions techniques individuelles.

Critère	Barème	Éléments évalués concrètement
Qualité des supports visuels	/ 3	Diapositives claires et professionnelles, schémas d'architecture soignés, absence de blocs de code illisibles.

Critère	Barème	Éléments évalués concrètement
Démonstration technique live	/ 4	Démonstration fluide de l'application Streamlit sans plantage imprévu, exécution en direct de la suite de tests <code>pytest</code> .
Maîtrise individuelle du code	/ 8	Aptitude de chaque étudiant à naviguer dans le code, à expliquer précisément la logique d'une fonction et à justifier ses choix algorithmiques.
Compréhension des concepts	/ 3	Maîtrise rigoureuse du vocabulaire : polymorphisme, complexité temporelle, stochasticité, décorateurs, conditions de concurrence.
Investissement Git vérifiable	/ 2	Cohérence vérifiée entre les déclarations du fichier <code>CONTRIBUTIONS.md</code> et les métriques réelles d'activité GitHub (commits, PRs créées et revues).

11 Calendrier Prévisionnel et Jalons de Développement

Le calendrier est articulé autour de jalons structurants afin d'assurer une progression continue :

- **Fin Septembre (Séance 2) :**
 - Constitution définitive des trinômes.
 - Inscription sur le tableur partagé pour l'attribution de l'extension thématique (règle du premier arrivé, premier servi).
- **Mi-Octobre (Jalon 1) :**
 - Initialisation du dépôt GitHub à partir du template officiel.
 - Mise en place de la protection de branche sur `main` et vérification de la CI GitHub Actions.
- **Fin Octobre (Jalon 2) :**
 - Finalisation du cœur orienté objet dans `src/pyarena/core/` (`BaseFighter`, `Fighter`, `engine.py`).
 - Implémentation des premières exceptions personnalisées et validation des tests unitaires `pytest` sur les règles de combat de base.
- **Mi-Novembre (Jalon 3) :**
 - Client PokéAPI opérationnel avec politique de mise en cache locale sur disque (`data/cache/`).
 - Moteur de simulation Monte-Carlo parallélisé fonctionnel avec calcul des intervalles de confiance asymptotiques.
- **Début Décembre (Jalon 4) :**

- Implémentation et tests unitaires de l’extension thématique finalisés.
- Dashboard Streamlit achevé (simulateur live, catalogue et cartographie Folium des arènes régionales).
- **Mi-Décembre :**
 - Gel définitif des dépôts GitHub sur la branche `main`.
 - Soutenances orales de fin de semestre.